Задание 1 · Что требуется и как загружаются данные

Нужно: сравнить словарный и ML-метод тональности: negative / neutral / positive; подготовить train/validation/test, метрики, ошибки и графики. **Дано:** текст задания; датасет нужно найти. Взяты отзывы Twitter US Airline Sentiment: все три метки уже размечены. CSV скачивается из `martinpella/twitter-airlines`, без Kaggle-токена.

Как читать документ: слева настоящий код с номерами строк исходного блока, справа — что делает каждая команда; ниже — сохранённый результат и его смысл. Перенесённые строки одного выражения объясняются вместе. Блоки считаются только среди 21 кодовой ячейки. Длинные тексты, списки импортов/хешей и повторные печать/сохранение сокращены, вычислительные шаги разобраны. Основы синтаксиса Python отдельно не пересказываются.

Общая подготовка, блоки 1–2: pip ставит библиотеки текущему Python. NumPy/pandas хранят массивы/таблицы, NLTK обрабатывает текст, sklearn обучает и оценивает модели, matplotlib/WordCloud строят графики. SEED=42 фиксирует split/выборки; DATA/RESULTS — кеш/результаты. NLTK скачивает punkt_tab, WordNet, POS-теггер, stopwords и VADER. Классификаторы ещё не обучены.

Блок 2, строки 47–65

```
47 def fetch(url, path, expected_sha256=None):
48     """HTTPS + атомарная запись; при заданном хеше проверяется также
кеш."""
49     path = Path(path); path.parent.mkdir(parents=True,
exist_ok=True)
50     if path.exists() and expected_sha256 and sha256_file(path) !=
expected_sha256:
51         raise ValueError(f'Контрольная сумма кеша не совпала:
{path}. Удалите этот файл и повторите.')
52     if not path.exists():
53         tmp = path.with_suffix(path.suffix + '.part')
54         with session.get(url, stream=True, timeout=(20, 180)) as
response:
55             response.raise_for_status()
56             with open(tmp, 'wb') as f:
57                 for block in response.iter_content(1024 * 1024):
58                     if block: f.write(block)
59         if expected_sha256 and sha256_file(tmp) != expected_sha256:
60             tmp.unlink(missing_ok=True)
61             raise ValueError(f'Контрольная сумма загрузки не
совпала: {url}')
62         tmp.replace(path)
63     if path.stat().st_size == 0: raise ValueError(f'Пустой файл:
{path}')
64     DOWNLOAD_LOG[str(path)] = {'url': url, 'sha256':
sha256_file(path)}
65     return path
```

47–49. `fetch(url,path,...)` получает адрес и место сохранения; создаёт родительскую папку.

50–51. Если задан ожидаемый SHA-256, проверяет уже существующий кеш; несовпадение останавливает работу.

52–53. При отсутствии файла выбирает временное имя .part.

54–58. Открывает HTTPS-запрос, проверяет HTTP-статус и пишет файл порциями по 1 MiB.

59–62. Проверяет известный хеш новой загрузки; повреждённую удаляет, правильную переименовывает.

63–65. Не принимает пустой файл; запоминает URL/хеш и возвращает путь для read_csv().

`sha256_file()` (блок 2, 41–45) считает отпечаток файла порциями. Сетевая session повторяет временные сбои до 3 раз. `save_table()` пишет CSV в UTF-8. У CSV фактический хеш записывается, но без expected_sha256 кеш не сверяется с независимым эталоном. **Выход fetch — путь, не готовая таблица**: pandas прочитает его на стр. 3.

Задание 1 · Предобработка: строка → леммы

Блок 3, строки 1–26

```
01 STOP_EN = set(stopwords.words('english'))
02 NEGATIONS = {'no', 'not', 'nor', 'never'}
03 LEMMATIZER = WordNetLemmatizer()
05 def english_tokens(text, keep_negations=True):
06     text = html.unescape(str(text)).lower().replace('’', "'")
07     text = re.sub(r"won't", 'will not', text)
08     text = re.sub(r"can't", 'can not', text)
09     text = re.sub(r"n't\b", ' not', text)
10     text = re.sub(r'https?://\S+|www\.\S+|@\w+', ' ', text)
11     words = re.findall(r'[a-z]+', text)
12     tagged = nltk.pos_tag(words)
13     removed = STOP_EN - NEGATIONS if keep_negations else STOP_EN
14     pos_map = {'J': wordnet.ADJ, 'V': wordnet.VERB, 'N':
wordnet.NOUN, 'R': wordnet.ADV}
15     return [LEMMATIZER.lemmatize(w, pos_map.get(tag[0],
wordnet.NOUN))
16             for w, tag in tagged if len(w) > 1 and w not in removed]
18 @lru_cache(maxsize=30000)
19 def sentiment_preprocess(text):
20     return ' '.join(english_tokens(text, keep_negations=True))
22 assert 'not' in english_tokens('The flights were not good!')
23 assert 'eye' in english_tokens('My eyes and hands')
24 display(pd.DataFrame({'original': ['The flights were not good!', 'My
eyes and hands'],
25                       'processed': [sentiment_preprocess(t) for t in
26                                     ['The flights were not good!',
'My eyes and hands']]}))
```

1–3. Берём английские стоп-слова, отдельно сохраняемые отрицания, создаём WordNet-лемматизатор.

5–6. Функция получает текст; декодирует HTML, делает lowercase, заменяет тип апострофа.

7–9. Раскрывает won’t/can’t/остальные n’t, чтобы не потерять not.

10. Удаляет ссылки и @аккаунты: это не слова для нашей ML-модели.

11. Выделяет английские слова. Пунктуация, цифры и прочие алфавиты пропадают.

12. POS-теггер определяет часть речи каждого слова.

13. Из списка удаляемых слов исключаются отрицания при keep_negations=True.

14. Переводит теги J/V/N/R в adjective/verb/noun/adverb WordNet.

15–16. Отбирает слова длиннее 1 буквы и вне stopwords, затем приводит к лемме. Неизвестный POS считается noun.

18–20. Кеширует до 30 000 строк; соединяет леммы пробелами для TF-IDF.

22–23. Проверяет, что not сохранено, eyes превратилось в eye.

24–26. Создаёт таблицу original/processed и показывает два примера.

Точный вывод этой ячейки:

```
The flights were not good!  →  flight not good
My eyes and hands           →  eye hand
```

Flights/eyes/hands объединяются с базовыми формами. The/were/my/and удаляются. Сохранённое not позволяет получить признак **not good**, а не перепутать жалобу с похвалой. Обработка рассчитана на английский; в задаче языка используется другая, Unicode-функция.

Задание 1 · CSV → чистые отзывы → три выборки

Блок 4, строки 1–19

```
01 AIRLINE_URL = ('https://raw.githubusercontent.com/martinpella/
twitter-airlines/'
02     '5ef288ca005842165851e9559fdffcdcfdf7e271/data/Airline-
Sentiment-2-w-AA.csv')
03 reviews = pd.read_csv(fetch(AIRLINE_URL, DATA /
'airline_reviews.csv'))
04 reviews = reviews[['text',
'airline_sentiment']].dropna().rename(columns={'airline_sentiment':
'label'})
05 SENTIMENT_LABELS = ['negative', 'neutral', 'positive']
06 reviews = reviews[reviews.label.isin(SENTIMENT_LABELS)].copy()
07 reviews['processed'] = reviews.text.map(sentiment_preprocess)
08 reviews = reviews[reviews.processed.str.len() > 0]
09 conflicting = reviews.groupby('processed').label.nunique()
10 reviews = reviews[~reviews.processed.isin(conflicting[conflicting >
1].index)]
11 reviews =
reviews.drop_duplicates('processed').reset_index(drop=True)
12 sent_train, sent_other = train_test_split(reviews, test_size=.4,
random_state=SEED, stratify=reviews.label)
13 sent_val, sent_test = train_test_split(sent_other, test_size=.5,
random_state=SEED, stratify=sent_other.label)
14 assert set(sent_train.processed).isdisjoint(sent_val.processed)
15 assert set(sent_train.processed).isdisjoint(sent_test.processed)
16 assert set(sent_val.processed).isdisjoint(sent_test.processed)
17 display(pd.concat({name: frame.label.value_counts() for name, frame
in
18     [('train', sent_train), ('validation', sent_val), ('test',
sent_test)]}, axis=1).fillna(0).astype(int))
19 save_table(reviews[['text', 'label']], 'sentiment_clean_data.csv')
```

1–3. Фиксированный Git-коммит в URL → fetch возвращает файл → read_csv создаёт таблицу reviews.

4. Оставляет текст/метку, удаляет пропуски, переименовывает airline_sentiment в label.

5–6. Оставляет только три разрешённых класса.

7. map вызывает предыдущую функцию для каждого text; создаёт processed.

8. Удаляет отзывы, от которых после очистки ничего не осталось.

9–10. Находит одинаковые processed с разными метками и исключает конфликтующие группы.

11. Удаляет повторы processed, заново нумерует строки.

12. Оставляет 60% train, 40% временно помещает в sent_other.

13. Делит эти 40% пополам: validation и test примерно по 20%.

14–16. Проверяет отсутствие одинаковых processed между всеми парами наборов.

17–19. Считает метки по наборам, показывает таблицу, сохраняет очищенные данные.

Результат:

Метка	Train	Validation	Test
negative	5 409	1 803	1 804
neutral	1 703	567	568
positive	1 276	426	425
Всего	8 388	2 796	2 797

`stratify=label` сохраняет близкие доли классов, `random_state=SEED` — воспроизводимый split. Text нужен VADER и человеку; processed — ML. Разные исходники после очистки могут стать одинаковыми, поэтому проверяется именно используемое моделью представление. **Утечка:** один отзыв в train и test способен завысить качество. TF-IDF тоже нельзя заранее обучать на всех наборах. Метрики относятся к очищенной совокупности: удаление спорных меток меняет исходную задачу и не делает разметку абсолютно безошибочной.

Задание 1 · Как выбираются пороги и обучается ML

Блок 5, строки 1–17

```
01 vader = SentimentIntensityAnalyzer()
02 def lexicon_scores(texts):
03     return np.array([vader.polarity_scores(str(t))['compound'] for t
in texts])
04 def classify_scores(scores, lower, upper):
05     return np.where(scores < lower, 'negative', np.where(scores >
upper, 'positive', 'neutral'))
07 val_scores = lexicon_scores(sent_val.text)
08 threshold_trials = []
09 for lower in [-.6, -.4, -.2, -.1, -.05, 0.0]:
10     for upper in [0.0, .05, .1, .2, .4, .6]:
11         if lower >= upper: continue
12         pred = classify_scores(val_scores, lower, upper)
13         threshold_trials.append({'lower': lower, 'upper': upper,
**evaluate(sent_val.label, pred)})
14 threshold_table =
pd.DataFrame(threshold_trials).sort_values('macro_f1', ascending=False,
kind='stable')
15 best_lower, best_upper = threshold_table.iloc[0][['lower',
'upper']].astype(float)
16 display(threshold_table.head())
17 print('Выбранные пороги:', best_lower, best_upper)
```

1–3. VADER получает готовый словарь; polarity_scores возвращает compound для каждого текста.

4–5. Ниже lower — negative, выше upper — positive, всё между ними — neutral.

7–11. Считает validation-score, перебирает пары порогов; lower≥upper пропускает.

12–13. Превращает score в классы; сохраняет пороги и метрики каждой пары.

14–17. Сортирует по macro F1, берёт лучшую пару, выводит её.

Вывод: `lower=0.0, upper=0.4`. −0.3 → negative; 0.2 → neutral; 0.7 → positive. Ровно 0 и 0.4 — neutral: сравнения строгие. Compound — оценка, не вероятность positive.

Блок 6, строки 1–17

```
01 def make_sentiment_model(C):
02     return Pipeline([
03         ('tfidf', TfidfVectorizer(ngram_range=(1, 2), min_df=2,
max_df=.98,
04                                   max_features=35000,
sublinear_tf=True)),
05         ('lr', LogisticRegression(C=C, class_weight='balanced',
max_iter=1200, random_state=SEED))
06     ])
08 ml_trials = []
09 for C in [.5, 1., 2.]:
10     candidate = make_sentiment_model(C)
11     candidate.fit(sent_train.processed, sent_train.label)
12     ml_trials.append({'C': C, **evaluate(sent_val.label,
candidate.predict(sent_val.processed))})
13 ml_validation = pd.DataFrame(ml_trials).sort_values('macro_f1',
ascending=False, kind='stable')
14 display(ml_validation)
15 best_C = float(ml_validation.iloc[0].C)
16 sent_final_train = pd.concat([sent_train, sent_val])
17 sent_model =
make_sentiment_model(best_C).fit(sent_final_train.processed,
sent_final_train.label)
```

1–6. Создаёт новый Pipeline: сначала TF-IDF, затем Logistic Regression.

3–4. Слова и пары слов; минимум 2 документа, максимум 98%; до 35 000 признаков; логарифмический TF.

5. Balanced компенсирует дисбаланс весом ошибок; C регулирует штраф, max_iter ограничивает оптимизатор.

8–12. Для каждого C обучает pipeline на train; predict оценивает validation.

13–15. Выбирает C с максимальным validation macro F1.

16–17. После выбора заново обучает TF-IDF и LR на train+validation; test не используется.

Validation: C=0.5 → F1 0.7129; C=1 → 0.7144; **C=2 → 0.7145**, выбрано 2. VADER анализирует исходный текст, сохраняя регистр/пунктуацию; ML — обработанные леммы. Pipeline.fit учит словарь, IDF и веса; predict применяет их без расширения словаря по test. Для каждого класса LR считает сумму «вес × TF-IDF» плюс смещение; softmax превращает оценки в вероятности, predict выбирает максимальную. Balanced меняет цену ошибок, не число отзывов.

Задание 1 · Проверка, реальные ошибки и новый отзыв

Блок 6, строки 19–32

```
19 lex_pred = classify_scores(lexicon_scores(sent_test.text),
best_lower, best_upper)
20 ml_pred = sent_model.predict(sent_test.processed)
21 majority = sent_final_train.label.mode()[0]
22 sent_metrics = pd.DataFrame([
23     {'method': 'Majority baseline', **evaluate(sent_test.label,
np.repeat(majority, len(sent_test)))},
24     {'method': 'VADER (validation thresholds)',
**evaluate(sent_test.label, lex_pred)},
25     {'method': 'TF-IDF + Logistic Regression',
**evaluate(sent_test.label, ml_pred)}
26 ])
27 display(sent_metrics)
28 save_table(sent_metrics, 'task1_metrics.csv')
29 for name, pred in [('VADER', lex_pred), ('TF-IDF + LR', ml_pred)]:
30     print(name)
31     print(classification_report(sent_test.label, pred,
labels=SENTIMENT_LABELS, zero_division=0))
32     show_confusion(sent_test.label, pred, SENTIMENT_LABELS, name)
```

19–20. Получает два массива ответов на test: VADER с выбранными порогами и итоговая LR.

21. Находит самый частый обучающий класс: negative.

22–26. Считает общий baseline и два метода: evaluate сравнивает label с ответами.

27–28. Показывает и сохраняет метрики.

29–32. По каждому методу печатает precision/recall/F1 и строит confusion matrix.

Метод, test n=2 797	Accuracy	Macro F1
Всегда negative	64.50%	0.2614
VADER	55.85%	0.5326
TF-IDF + LR	76.87%	0.7057

VADER ниже baseline по accuracy, но выше по macro F1: он распознаёт и редкие классы. LR лучше обоих; её F1 negative≈0.86, neutral≈0.57, positive≈0.69 — neutral сложнее.

Блок 7, строки 1–24

```
01 error_rows = []
02 for method, predictions in [('VADER', lex_pred), ('TF-IDF + LR',
ml_pred)]:
03     errors = sent_test.assign(predicted=predictions).query('label !=
predicted')
04     for row in errors.head(8).itertuples():
05         lower = row.text.lower()
06         hints = []
07         if re.search(r"\b(not|no|never)\b|n't", lower):
hints.append('проверить область действия отрицания')
08         if any(w in lower for w in ['thanks', 'great', 'love',
'awesome']):
09             hints.append('проверить иронию / смешанную тональность;
возможна неоднозначность разметки')
10         if not hints: hints.append('проверить контекст услуги,
многозначность и потери при предобработке')
11         error_rows.append({'method': method, 'text': row.text,
'true': row.label,
12                            'predicted': row.predicted,
'hypothesis_to_check': '; '.join(hints)})
13 sentiment_errors = pd.DataFrame(error_rows)
14 with pd.option_context('display.max_colwidth', 180):
display(sentiment_errors)
15 save_table(sentiment_errors, 'task1_errors.csv')
17 def predict_sentiment(text):
18     score = float(lexicon_scores([text])[0])
19     return {'text': text, 'lexicon_score': score,
20         'VADER': classify_scores(np.array([score]), best_lower,
best_upper)[0],
21         'ML': sent_model.predict([sentiment_preprocess(text)])[0]}
22 display(pd.DataFrame([predict_sentiment(t) for t in [
23     'I loved the friendly service!', 'My flight was cancelled and
nobody helped.',
24     'The flight departs at six.', 'Great, another three-hour
delay.']]))
```

1–4. Для каждого метода добавляет predicted, оставляет label≠predicted, берёт первые 8 ошибок.

5–10. Подсказывает проверить отрицание, иронию или контекст. Это гипотезы, не доказанные причины.

11–15. Сохраняет текст, истинную метку, ответ и подсказку; показывает и пишет CSV.

17–21. Для нового text возвращает compound и метки VADER/ML; [text] — коллекция из одного документа.

22–24. Запускает функцию на четырёх иллюстративных отзывах.

Реальный пример выхода: `Great, another three-hour delay.` → compound **0.4215**, VADER **positive**, ML **negative**. Great положительное, но задержка делает фразу жалобой. Это удачный пример ML, не доказательство понимания всего сарказма. Первых 8 ошибок — иллюстрации, а не случайная репрезентативная выборка.

Задание 1 · Визуализация и теория для защиты

Блок 8, строки 2–23

```
02 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
03 for ax, label in zip(axes, ['positive', 'negative']):
04     frequencies = Counter('
' .join(sent_final_train.loc[sent_final_train.label == label,
'processed']).split())
05     cloud = WordCloud(width=800, height=450,
background_color='white', random_state=SEED,
06
collocations=False).generate_from_frequencies(frequencies)
07     ax.imshow(cloud, interpolation='bilinear'); ax.axis('off');
ax.set_title(label)
08 fig.tight_layout(); fig.savefig(RESULTS / 'task1_wordclouds.png',
dpi=150); plt.show()
10 features = sent_model.named_steps['tfidf'].get_feature_names_out()
11 classifier = sent_model.named_steps['lr']
12 fig, axes = plt.subplots(1, 3, figsize=(17, 5))
13 weight_rows = []
14 for ax, label, weights in zip(axes, classifier.classes_,
classifier.coef_):
15     top = np.argsort(weights)[-12:]
16     ax.barh(features[top], weights[top]); ax.set_title(f'Weights for
{label}')
17     for j in top: weight_rows.append({'class': label, 'feature':
features[j], 'weight': weights[j]})
18 fig.tight_layout(); fig.savefig(RESULTS /
'task1_feature_weights.png', dpi=150); plt.show()
19 save_table(pd.DataFrame(weight_rows), 'task1_top_features.csv')
20 best = sent_metrics.iloc[1:].sort_values('macro_f1',
ascending=False).iloc[0]
21 display(Markdown(f'**Вывод:** на этом test лучше по macro F1 метод
**{best.method}** '
22     f'({best.macro_f1:.3f}). Для улучшения: учитывать область
отрицания, словарь авиаперевозок, '
23     'контекстные модели и отдельно оценивать сарказм. Перенос на
другие домены требует новой проверки.'))
```

2–4. Для positive/negative собирает токены только train+validation и считает Counter.

5–8. Строит WordCloud по частоте, показывает и сохраняет две картинки. Размер слова связан с числом вхождений.

10–11. Достаёт имена TF-IDF-признаков и обученную LR.

12–14. Создаёт три графика и сопоставляет класс его строке coef_.

15. argsort возвращает индексы весов; [-12:] берёт 12 самых больших.

16–19. Рисует веса по признакам, собирает таблицу и сохраняет её.

20–23. Из двух методов выбирает лучший test macro F1 и выводит итоговый комментарий.

Как получается график: словарь частот → размер слова в облаке; `coef_` → длина столбца весов. Частота и вес — разные величины: частое в обоих классах слово мало помогает различать классы. Большой вес — ассоциация с классом, не доказательство причинности.

Контрольные вопросы, относящиеся к задаче

КВ 6. Словарь и ML: VADER использует готовые оценки/правила; LR учит веса на размеченных отзывах. Пороги VADER всё же настроены по validation. **КВ 7. Леммы:** объединяют формы слов; POS помогает выбрать нужную. Stemming обычно обрезает окончания.

КВ 8–9. Матрица и accuracy: строки — истина, столбцы — ответ; диагональ верна. Accuracy — доля правильных ответов, она маскирует дисбаланс. TP — верно найденный класс, FP — ошибочно названный этим классом, FN — пропущенный. Precision=TP/(TP+FP), recall=TP/(TP+FN), F1=2PR/(P+R); macro F1 усредняет классы поровну. **КВ 10:** мешают сарказм, отрицания, смешанная тональность, домен и шумная разметка.

КВ 11–12,15. Признаки: BoW — количества слов без порядка; биграммы добавляют локальный порядок. TF-IDF понижает роль повсеместных слов: `idf=log((1+N)/(1+df))+1`; sublinear TF=`1+log(count)`, затем L2-норма. Это не z-score-стандартизация `(x−mean)/std`. **КВ 14:** ROC сравнивает TPR/FPR, PR — precision/recall; при редком positive PR часто полезнее. В ноутбуке кривые не строились.

Дополнительно: меньше C — сильнее регуляризация; balanced не добавляет данных. Train учит, validation выбирает настройки, test проверяет результат. Тест не используют для бесконечной донастройки. Веса — параметры модели; C и n-граммы — гиперпараметры.

Задание 2 · Условие, статьи и предложения

Дано: 5 текстовых файлов. **Нужно:** из первых 3 выбрать по 3 предложения и общий top-6; по всем 5 сгенерировать короткую связную аннотацию и оценить её. Датасет искать не нужно. В блоке 9 `EMBEDDED_ARTICLES` хранит исходники как строки, а не выполняемые инструкции.

Блок 10, строки 1–26

```
01 def clean_article(text):
02     text = unicodedata.normalize('NFKC', text).replace('\x02', '')
03     text = re.sub(r'(?<=\w)-\s*\n\s*(?=\w)', '', text)
04     text = re.sub(r'^\s*(?:\d+\.\s*)?(?:Introduction|Rhythm
Plagiarism)\s*$', '', text, flags=re.M)
05     text = re.sub(r'\[\d+\]', '', text)
06     return re.sub(r'\s+', ' ', text).strip()
08 documents = {}
09 for name, embedded in EMBEDDED_ARTICLES.items():
10     local_path = Path(name)
11     original = local_path.read_text(encoding='utf-8-sig') if
local_path.exists() else embedded
12     documents[name] = clean_article(original)
13 display(pd.DataFrame([{'document': name, 'words': len(text.split()),
14     'sentences': len(nltk.sent_tokenize(text))} for name, text in
documents.items()]))
16 extractive_docs = list(documents)[:3]
17 sentence_rows = []
18 for doc in extractive_docs:
19     sentences = list(dict.fromkeys(s.strip() for s in
nltk.sent_tokenize(documents[doc]) if len(s.split()) >= 5))
20     assert len(sentences) >= 3, f'Слишком мало предложений: {doc}'
21     sentence_rows.extend({'document': doc, 'position': i,
'sentence': sentence}
22                          for i, sentence in enumerate(sentences))
23 sentence_frame = pd.DataFrame(sentence_rows)
24 sentence_frame['sentence_key'] = sentence_frame.sentence.map(
25     lambda s: re.sub(r'[^a-z0-9]+', ' ', s.lower()).strip())
26 sentence_frame = sentence_frame.drop_duplicates(['document',
'sentence_key']).reset_index(drop=True)
```

1–6. clean_article: NFKC, удаление управляющего символа, разорванных слов, некоторых заголовков, [ссылок], лишних пробелов.

8–12. Для каждой статьи выбирает локальный файл, если он есть; иначе встроенную строку. Результат чистит.

13–14. Считает слова через split и предложения через sent_tokenize; выводит размеры.

16–18. Берёт первые три имени, создаёт список предложений по документам.

19. Делит на предложения, оставляет минимум 5 слов, удаляет точные повторы, сохраняя порядок.

20–22. Проверяет наличие хотя бы 3 предложений; записывает документ, позицию и текст.

23–26. Создаёт таблицу, нормализует ключ без регистра/пунктуации, убирает такие повторы внутри документа.

Точный результат подсчёта:

Документ	Слова	Предложения
article1	426	33
article2	699	36
article3	636	33
article4	527	30
article5	314	18

Article3 содержит повтор abstract: удаление повторов снижает его лишний вклад. Первые 3 статьи описывают сравнение мелодий, быстрый поиск и аудиоотпечатки. **Мы обрабатываем тексты об аудио, а не вычисляем аудиопризнаки:** MFCC/энергия/энтропия здесь упоминаются в источниках. Ограничение очистки — заголовки и границы предложений могут распознаваться неидеально.

Задание 2 · TF-IDF → важность → извлечение

Блок 10, строки 28–50

```
28 sentence_vectorizer = TfidfVectorizer(stop_words='english',
ngram_range=(1, 2), sublinear_tf=True)
29 sentence_X =
sentence_vectorizer.fit_transform(sentence_frame.sentence)
30 centroid = np.asarray(sentence_X.mean(axis=0))
31 sentence_frame['score'] = cosine_similarity(sentence_X,
centroid).ravel()
33 per_document = {}
34 for name in extractive_docs:
35     top = sentence_frame[sentence_frame.document ==
name].nlargest(3, 'score').sort_values('position')
36     per_document[name] = top.sentence.tolist()
37     display(Markdown(f'### {name}: 3 ключевых предложения'))
38     for i, sentence in enumerate(top.sentence, 1): print(f'{i}.
{sentence}')
42 combined_six = (sentence_frame.sort_values('score', ascending=False,
kind='stable')
43     .drop_duplicates('sentence_key').head(6).reset_index(drop=True))
44 assert len(combined_six) == 6
45 display(Markdown('### Единый список: 6 предложений по убыванию
веса'))
46 for i, row in enumerate(combined_six.itertuples(), 1):
47     print(f'{i}. [{row.document}; score={row.score:.3f}]
{row.sentence}')
48 save_table(sentence_frame, 'task2_sentence_scores.csv')
49 save_table(combined_six, 'task2_combined_six.csv')
50 (RESULTS /
'task2_extractive.json').write_text(json.dumps(per_document,
ensure_ascii=False, indent=2), encoding='utf-8')
```

28–29. Общий TF-IDF слов/биграмм предложений трёх статей. fit_transform учит признаки и сразу строит матрицу.

30. Среднее по строкам: центроид — типичная общая тема.

31. Косинус каждого предложения с центроидом становится score; ravel делает одномерный массив.

33–36. В каждой статье берёт 3 максимальных score; sort_values(position) возвращает их исходный порядок.

37–38. Печатает заголовок и выбранные предложения.

42–44. Сортирует общий набор, удаляет совпадающие sentence_key между статьями, берёт 6; проверяет количество.

45–47. Выводит предложения с источником и score.

48–50. Сохраняет все оценки, общий top-6 и сводки отдельных документов.

Примеры реального вывода: article1 включает “When music has similar melody progress with different key and instruments, plagiarism can be suspected.” Article2 — “Similar melody searching is to find the melodies similar to a given query melody from a music database.” Article3 — “Music plagiarism is not a new phenomena.” Всего выбрано 3×3 предложения.

Первый пункт общего top-6 — предложение о similar melody searching из article2, score≈**0.324**. Общий результат — **137 слов**; после удаления нормализованных совпадений остаются и близкие по смыслу фразы. Выбор не обязан давать по 2 предложения от каждой статьи.

Почему такой выход: $\cos(a,b)=a\cdot b/(||a||\cdot||b||)$ измеряет общность признаков. Он не оценивает научный вклад: общая лексика может поднять фразу об оглавлении. Здесь выбираются исходные предложения, а не генерируются новые.

Задание 2 · Загрузка T5 и преобразование в токены

Блок 11: T5_CHECKSUMS сопоставляет файлы ожидаемым SHA-256. Config описывает модель, tokenizer/spiece — словарь токенов, ONNX — вычислительные графы и веса. Длинные хеши — данные для fetch, а не языковые параметры. Использована публичная копия T5-small в INT8.

Блок 12, строки 1–26

```
01 import torch
02 import onnxruntime as ort
03 from transformers import AutoTokenizer
04 from optimum.onnxruntime import ORTModelForSeq2SeqLM
06 T5_BASE = 'https://raw.githubusercontent.com/Arfius/t5-small/
88f269658b785fecf015c2f787a3fa9486d349f4/'
07 T5_DIR = DATA / 't5_small_onnx'
08 for filename, digest in T5_CHECKSUMS.items():
09     fetch(T5_BASE + filename, T5_DIR / filename,
expected_sha256=digest)
11 tokenizer = AutoTokenizer.from_pretrained(str(T5_DIR),
local_files_only=True)
12 ort_options = ort.SessionOptions()
13 ort_options.intra_op_num_threads = min(4, os.cpu_count() or 1)
14 ort_options.inter_op_num_threads = 1
15 summary_model = ORTModelForSeq2SeqLM.from_pretrained(str(T5_DIR),
16     encoder_file_name='onnx/encoder_model_quantized.onnx',
17     decoder_file_name='onnx/decoder_model_merged_quantized.onnx',
18     use_merged=True, use_cache=True, local_files_only=True,
19     session_options=ort_options, provider='CPUExecutionProvider')
21 def token_count(text): return len(tokenizer.encode(text,
add_special_tokens=False))
23 def token_chunks(text, size=430):
24     ids = tokenizer.encode(text, add_special_tokens=False)
25     return [tokenizer.decode(ids[i:i+size],
skip_special_tokens=True)
26             for i in range(0, len(ids), size)]
```

1–4. PyTorch — тензоры; ONNX Runtime/Optimum — выполнение; AutoTokenizer — текст↔токены.

6–9. Для каждого файла строит URL закреплённого коммита и скачивает с проверкой контрольной суммы.

11. Загружает токенизатор с диска; local_files_only запрещает дополнительную сетевую загрузку.

12–14. Ограничивает параллелизм CPU максимум 4 потоками внутри операции и 1 между операциями.

15–19. Создаёт готовую encoder–decoder модель на CPU. use_cache переиспользует вычисления внимания при генерации.

21. encode превращает строку в IDs; len считает токены без служебных маркеров.

23–24. Получает все токены входа.

25–26. Режет IDs по 430 и декодирует каждый отрезок обратно в текст.

Результат ячейки: созданы tokenizer и summary_model; отдельный текст сводки пока не печатался. Из длинной статьи получится список блоков для следующего шага. Один токен может быть частью слова, поэтому 430 токенов ≠ 430 слов.

T5 имеет encoder, который представляет вход, и decoder, который пишет продолжение. Attention связывает выход с контекстом. ONNX — формат выполнения, не другая архитектура. INT8 экономит память; его отдельное влияние на качество не измерялось. Обучения T5 здесь нет.

Задание 2 · Как именно получается новая сводка

Блок 12, строки 28–62

```
28 def summarize_chunk(text, max_new_tokens=85):
29     inputs = tokenizer('summarize: ' + text, return_tensors='pt',
truncation=False)
30     assert inputs['input_ids'].shape[1] <= 512, 'Ошибка разбиения на
токеновые блоки'
31     with torch.inference_mode():
32         generated = summary_model.generate(**inputs,
max_new_tokens=max_new_tokens,
33             min_new_tokens=20, num_beams=4, do_sample=False,
34             no_repeat_ngram_size=3, repetition_penalty=1.1,
early_stopping=True)
35     return tokenizer.decode(generated[0],
skip_special_tokens=True).strip()
37 document_digests = {}
38 chunk_log = []
39 for name, text in documents.items():
40     chunks = token_chunks(text)
41     summaries = []
42     for i, chunk in enumerate(chunks):
43         summary = summarize_chunk(chunk, max_new_tokens=75)
44         summaries.append(summary)
45         chunk_log.append({'document': name, 'chunk': i, 'tokens':
token_count(chunk), 'summary': summary})
46     document_digests[name] = ' '.join(summaries)
47     print(name, '→', len(chunks), 'блоков обработано')
49 merged = ' '.join(document_digests.values())
50 for _ in range(8):
51     if token_count(merged) <= 430: break
52     smaller = ' '.join(summarize_chunk(c, max_new_tokens=65) for c
in token_chunks(merged))
53     if token_count(smaller) >= token_count(merged):
54         raise RuntimeError('Сводка не сокращается: необходимо
уменьшить max_new_tokens.')
55     merged = smaller
56 assert token_count(merged) <= 430
57 abstractive_summary = summarize_chunk(merged, max_new_tokens=150)
58 assert abstractive_summary and len(abstractive_summary.split()) <
sum(len(t.split()) for t in documents.values())
59 display(Markdown('### Сгенерированная абстрактивная сводка
(английский)'))
60 print(abstractive_summary)
61 save_table(pd.DataFrame(chunk_log), 'task2_chunk_summaries.csv')
62 (RESULTS / 'task2_abstractive.txt').write_text(abstractive_summary,
encoding='utf-8')
```

28–30. Добавляет задачу summarize:, создаёт тензоры, проверяет вход≤512. truncation=False не даёт молча обрезать документы.

31–35. Без градиентов генерирует продолжение, декодирует первый ответ без служебных токенов.

37–41. Для всех 5 документов создаёт части и список их сводок.

42–47. Сжимает каждую часть максимум до 75 новых токенов; пишет журнал и объединяет по документу.

49–52. Объединяет сводки; если длиннее 430 токенов, сжимает порциями снова.

53–56. Останавливает несокращающийся результат; максимум 8 уровней; проверяет итоговую длину.

57–58. Генерирует финал до 150 токенов; проверяет непустоту и меньшую длину, чем у корпуса.

59–62. Показывает и сохраняет итог; отдельно сохраняет сводки частей.

Вывод по частям: article1→2, article2→3, article3→2, article4→2, article5→1. **Точный финальный текст:**

> melody pitch sequence is converted into a note sequence. similarity between input polyphonic music and those of music in the database is calculated. break originated from the 1969 Funk recording Amen Brother by The Winstons.

Это **35 слов**. Прохождение всех документов через модель не гарантировало полноту: остались преобразование мелодии, сходство и происхождение фрагмента; важные темы потеряны. `num_beams=4` сохраняет несколько перспективных продолжений; `do_sample=False` убирает случайный sampling. `no_repeat_ngram_size=3` и penalty=1.1 снижают повторения, но не проверяют факты. Предупреждение encode о длинном тексте относится к нарезке; перед реальной моделью есть assert входа≤512. Последнее число от write_text — число символов. `min_new_tokens=20` запрещает слишком раннее завершение; `early_stopping=True` останавливает beam search по условию готовности кандидатов. Это не гарантия качества текста.

Задание 2 · ROUGE, покрытие и проверка фактов

В блоке 13 строки 2–20 — два **авторских учебных эталона**, для 3 и 5 статей. Они заданы текстом, не созданы моделью. Это не независимая экспертная разметка.

Блок 13, строки 21–43

```
21 extractive_text = ' '.join(combined_six.sentence)
22 scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
use_stemmer=True)
23 quality_rows = []
24 for method, summary, reference, names in [
25     ('Extractive (3 documents)', extractive_text, REFERENCE_THREE,
extractive_docs),
26     ('Abstractive (5 documents)', abstractive_summary,
REFERENCE_FIVE, list(documents))]:
27     scores = scorer.score(reference, summary)
28     quality_rows.append({'method': method, 'words':
len(summary.split()),
29         'compression_ratio': len(summary.split()) /
sum(len(documents[n].split()) for n in names),
30         **{f'{k}_F1': v.fmeasure for k, v in scores.items()}})
31 quality = pd.DataFrame(quality_rows)
32 display(quality); save_table(quality, 'task2_quality_metrics.csv')
34 coverage_rows = []
35 for method, summary in [('extractive_3_docs', extractive_text),
('abstractive_5_docs', abstractive_summary)]:
36     names = extractive_docs if method.startswith('extractive') else
list(documents)
37     vec = TfidfVectorizer(stop_words='english')
38     matrix = vec.fit_transform([documents[n] for n in names] +
[summary])
39     similarities = cosine_similarity(matrix[:-1],
matrix[-1]).ravel()
40     coverage_rows.extend({'method': method, 'document': n,
'lexical_coverage_proxy': float(v)}
41                          for n, v in zip(names, similarities))
42 coverage = pd.DataFrame(coverage_rows)
43 display(coverage); save_table(coverage,
'task2_document_coverage.csv')
```

21–22. Склеивает top-6; создаёт ROUGE-1/2/L со stemming, который смягчает различия форм слов.

23–27. Для двух кандидатов выбирает соответствующий эталон; score получает сначала reference, затем summary.

28–30. Считает слова, долю длины исходников и F1 каждого ROUGE.

31–32. Показывает и сохраняет таблицу качества.

34–38. В новом TF-IDF-пространстве представляет источники и их сводку.

39–43. Считает сходство каждого документа со сводкой; сохраняет лексический прокси покрытия.

Блок 13, строки 45–59

```
45 source_sentences = [(name, sentence) for name, text in
documents.items()
46                     for sentence in nltk.sent_tokenize(text) if
len(sentence.split()) >= 5]
47 generated_sentences = nltk.sent_tokenize(abstractive_summary)
48 ground_vec = TfidfVectorizer(stop_words='english').fit(
49     [s for _, s in source_sentences] + generated_sentences)
50 source_X = ground_vec.transform([s for _, s in source_sentences])
51 support_rows = []
52 for sentence in generated_sentences:
53     similarity = cosine_similarity(ground_vec.transform([sentence]),
source_X).ravel()
54     best = int(similarity.argmax())
55     support_rows.append({'generated': sentence, 'closest_source':
source_sentences[best][0],
56         'source_sentence': source_sentences[best][1],
'similarity_proxy': float(similarity[best])})
57 support = pd.DataFrame(support_rows)
58 with pd.option_context('display.max_colwidth', 220):
display(support)
59 save_table(support, 'task2_factual_support_review.csv')
```

45–47. Собирает исходные предложения и предложения сгенерированного текста.

48–50. Строит общий словарь и матрицу источников.

51–54. Для каждого предложения сводки ищет самое похожее исходное: argmax возвращает индекс.

55–59. Показывает пару, источник и сходство для ручной проверки; пишет CSV.

Фактический результат:

Кандидат	Слов / доля длины	R-1 F1	R-2 F1	R-L F1
Extractive, 3 статьи	137 / 7.78%	0.3153	0.0364	0.1892
T5, 5 статей	35 / 1.35%	0.1818	0.0000	0.0979

Например, фраза о melody pitch сопоставлена статье1: её модулю Melody-to-MIDI. Лексическое сходство помогает найти опору, но **не доказывает логическое следование**. Разные входы/эталоны не позволяют считать эти два ROUGE прямым честным турниром.

Задание 2 · Что означает результат и что спросит профессор

Связь результата с кодом

`max_new_tokens` только ограничивает длину. Он не задаёт список обязательных тем. Повторное сжатие кратких промежуточных текстов в блоке 12 теряет часть содержания. Поэтому факт обработки всех 10 частей не означает полного покрытия пяти статей.

В итоговом выводе T5 отсутствуют **MFCC/энтропия/энергия**, индексирование и сэмплирование; утрачено имя Amen перед break. Строчные начала и слабые переходы ухудшают оформление. Просто уменьшить длину недостаточно для выполнения требований полноты/связности/корректности.

После блока 13 в ноутбуке есть отдельная **ручная вычитанная аннотация**. Она объединяет:

Источник	Что важно сохранить
article1	PCM → мелодия → ноты MIDI → сравнение фрагментов
article2	Alignment/shifting, индексирование и быстрый поиск
article3	Аудиоотпечатки: MFCC, энтропия и энергия
article4	Сэмплирование и идентификация записи по отпечаткам
article5	Ритмическое/мелодическое сходство, Amen Break

Это редакторский результат, **не скрытая замена output модели**. ROUGE в таблице рассчитан на настоящей генерации. Защищать нужно выполненный эксперимент и диагностику, а не утверждать, что маленькая T5 автоматически дала качественную финальную аннотацию.

Контрольные вопросы этой задачи

КВ 1. Extractive / abstractive? Первый выбирает исходные предложения; второй генерирует новый текст. Извлечение может потерять контекст, генерация — ещё и придумать неверный факт.

КВ 2. Почему ранжируют предложения? Они выбраны единицами сводки: слова задают признаки, но отдельные слова ещё не образуют текст. TF-IDF сам по себе не ограничен предложениями. TextRank использовал бы граф сходства; у нас центроид, графового ранжирования нет.

КВ 3. Сложности генерации? Длинный вход, выбор важного, объединение источников, связность и галлюцинации. Нарезка может обрывать предложения, иерархия усиливает потери.

КВ 4. ROUGE? R-1 — слова, R-2 — пары, R-L — общая подпоследовательность с сохранением порядка. Это лексическая оценка: правильный парафраз может иметь мало совпадений, а похожая ложная фраза — много. Нулевой R-2 не доказывает ложность всех фактов.

КВ 5 и 13. Грамматика? Неверное отрицание/местоимение меняет смысл. Генератор выбирает вероятные токены, не проверяет строгую грамматическую теорему. Влияют шум, домен, размер модели и декодирование; отдельный ущерб от INT8 здесь не измерен.

Что улучшить: MMR/квоты и фильтр оглавления для извлечения; более сильная модель, разбиение по предложениям и проверка покрытия для генерации; независимые эталоны. MFCC не вычислялись: музыкальные методы здесь — содержание статей, не наша реализация.

Задание 3 · Условие, выбор песен и top-10

Нужно: ≥4 жанра, ≥10 песен каждого; top-10 слов, тематические категории, редкие слова. **Данных не было:** найден `genre_lyrics_data.csv` проекта etarakci/music-genre-prediction: тексты AZLyrics + жанры Spotify. Жанр группирует данные; классификатор не обучается.

Блок 14, строки 1–29

```
01 LYRICS_URL = ('https://raw.githubusercontent.com/etarakci/music-
genre-prediction/'
02     'cd007c16892ff98b40f8df1e33d26e3c8cf80127/data/
genre_lyrics_data.csv')
03 lyrics_all = pd.read_csv(fetch(LYRICS_URL, DATA /
'genre_lyrics_data.csv'))
04 lyrics_all = lyrics_all.rename(columns={'genre_name': 'genre',
'lyrics_cleaned': 'lyrics'})
05 lyrics_all['source_row'] = np.arange(len(lyrics_all))
06 lyrics_all = lyrics_all.dropna(subset=['genre', 'lyrics']).copy()
07 lyrics_all['canonical'] = lyrics_all.lyrics.map(lambda t:
re.sub(r'[^a-z]+', ' ', str(t).lower()).strip())
08 lyrics_all = lyrics_all[lyrics_all.canonical.str.split().str.len()
>= 30].drop_duplicates('canonical')
09
display(lyrics_all.genre.value_counts().head(20).to_frame('available_so
ngs'))
10 GENRES = ['country', 'blues', 'folk', 'hip-hop']
11 missing = [g for g in GENRES if (lyrics_all.genre == g).sum() < 10]
12 assert not missing, f'Не хватает 10 уникальных песен в жанрах
{missing}. Доступные: {lyrics_all.genre.unique()}'
13 lyrics_sample = pd.concat([lyrics_all[lyrics_all.genre ==
g].sample(10, random_state=SEED)
14                           for g in GENRES], ignore_index=True)
15 lyrics_sample['tokens'] = lyrics_sample.lyrics.map(lambda t:
english_tokens(t, keep_negations=False))
16 lyrics_sample['text_sha256'] = lyrics_sample.canonical.map(lambda t:
hashlib.sha256(t.encode()).hexdigest())
17 assert len(lyrics_sample) == 40 and
lyrics_sample.text_sha256.nunique() == 40
18 assert (lyrics_sample.groupby('genre').size() == 10).all()
19 save_table(lyrics_sample[['source_row', 'genre', 'text_sha256']],
'task3_selected_songs.csv')
20 genre_counts = {genre: Counter(w for words in
lyrics_sample.loc[lyrics_sample.genre == genre, 'tokens']
21                                for w in words) for genre in GENRES}
22 top10 = pd.DataFrame([{'genre': genre, 'word': word, 'count': count}
23     for genre, counts in genre_counts.items() for word, count in
counts.most_common(10)])
24 display(top10); save_table(top10, 'task3_top10.csv')
25 fig, axes = plt.subplots(2, 2, figsize=(13, 9))
26 for ax, genre in zip(axes.flat, GENRES):
27     top = genre_counts[genre].most_common(10)[::-1]
28     ax.barh([w for w, _ in top], [c for _, c in top]);
ax.set_title(genre)
29 fig.tight_layout(); fig.savefig(RESULTS / 'task3_top10.png',
dpi=150); plt.show()
```

1–4. Скачивает CSV закреплённого коммита; переименовывает колонки в genre и lyrics.

5. Запоминает исходный номер строки — в CSV нет названия/исполнителя.

6–8. Убирает пропуски, создаёт нормализованный ключ; оставляет ≥30 слов, удаляет повторы.

9–12. Показывает доступность жанров; проверяет, что в каждом выбранном есть хотя бы 10 текстов.

13–14. Выбирает по 10 с фиксированным seed, объединяет таблицы.

15. Вызывает английскую предобработку; здесь отрицания не сохраняет отдельно.

16–19. Хеширует нормализованные тексты; проверяет 40 разных и 10 на жанр, сохраняет идентификаторы.

20–21. Объединяет токены песен каждого жанра и считает Counter.

22–24. Берёт most_common(10), создаёт и сохраняет top-10.

25–29. Рисует четыре barh-графика; разворачивает список для удобного порядка по вертикали.

Выход: country=10, blues=10, folk=10, hip-hop=10. Первые три слова каждого top-10:

Жанр	Реальные частоты
Country	know 52; im 31; get 24
Blues	oh 73; love 17; im 16
Folk	get 31; love 25; night 18
Hip-hop	im 65; feel 52; like 45

График отображает именно эти счётчики. Источник уже очищен: im/dont/cant без апострофов не исправляются нашими правилами для can't. Это ограничение, а не ошибка Counter.

Задание 3 · Словари категорий → упоминания

Блок 15, строки 1–24

```
02 CATEGORIES = {
03     'time_of_day': {'morning', 'day', 'afternoon', 'evening',
'night', 'midnight', 'dawn', 'dusk'},
04     'seasons': {'winter', 'spring', 'summer', 'autumn', 'fall'},
05     'body_parts': {'eye', 'heart', 'hand', 'head', 'face', 'arm',
'leg', 'foot', 'hair',
06                    'mouth', 'lip', 'ear', 'finger', 'skin', 'body',
'blood', 'bone'}
07 }
08 category_rows = []
09 for genre, counts in genre_counts.items():
10     total = sum(counts.values())
11     for category, dictionary in CATEGORIES.items():
12         for word in sorted(dictionary):
13             n = counts[word]
14             category_rows.append({'genre': genre, 'category':
category, 'word': word,
15                                   'count': n, 'per_1000_tokens':
1000*n/total})
16 category_words = pd.DataFrame(category_rows)
17 display(category_words.query('count > 0').sort_values(['genre',
'category', 'count'], ascending=[True, True, False]))
18 category_totals = category_words.groupby(['genre', 'category'])
[['count', 'per_1000_tokens']].sum().reset_index()
19 display(category_totals)
20 category_totals.pivot(index='genre', columns='category',
values='per_1000_tokens').plot.bar(figsize=(11, 4))
21 plt.ylabel('Mentions per 1,000 processed tokens');
plt.xticks(rotation=0)
22 plt.tight_layout(); plt.savefig(RESULTS / 'task3_categories.png',
dpi=150); plt.show()
23 save_table(category_words, 'task3_category_words.csv')
24 save_table(category_totals, 'task3_category_totals.csv')
```

2–7. Три категории задаются множествами лемм: время суток, сезоны, части тела.

8–10. Для жанра берёт Counter и общее число обработанных токенов.

11–13. Для каждого слова категории читает количество; отсутствующее слово Counter даёт 0.

14–15. Сохраняет слово, жанр, категорию, количество и 1000×count/total.

16–17. Показывает только ненулевые слова; сортирует по жанру, категории и частоте.

18–19. Суммирует упоминания всех слов категории внутри жанра.

20. pivot превращает категории в колонки для групповой диаграммы.

21–24. Подписывает шкалу на 1000 токенов, показывает/сохраняет PNG и две таблицы.

Результат: ниже `count / на 1000 обработанных токенов.`

Жанр	Body parts	Seasons	Time of day
Blues	14 / 17.63	0 / 0	7 / 8.82
Country	16 / 12.36	2 / 1.55	15 / 11.59
Folk	33 / 29.84	7 / 6.33	23 / 20.80
Hip-hop	15 / 6.88	4 / 1.83	12 / 5.50

Для eye в blues count=8 и частота≈10.08/1000; eyes перед подсчётом стала eye. Без лемматизации две формы считались бы разными. При сравнении жанров знаменатель — **все оставшиеся токены**, не число песен и не только слова словарей.

Почему нужна относительная частота: 10 упоминаний среди 1000 и 2000 токенов — одинаковые count, но интенсивность 10 и 5 на тысячу. Равное число песен не делает равными длины их текстов.

Задание 3 · Редкие слова, результат и теория

Блок 15, строки 26–47

```
27 rare_per_genre = pd.DataFrame([{'genre': genre, 'word': word,
'count': count}
28     for genre, counts in genre_counts.items() for word, count in
sorted(counts.items()) if 1 <= count <= 3])
29 display(Markdown('### Редкие слова отдельно для каждого жанра (1–3
вхождения)'))
30
display(rare_per_genre.groupby('genre').size().to_frame('number_of_rare
_words'))
31 display(rare_per_genre.groupby('genre', sort=False).head(15))
32 save_table(rare_per_genre, 'task3_rare_per_genre_FULL.csv')
34 shared_vocabulary = set.intersection(*(set(c) for c in
genre_counts.values()))
35 shared_rare_words = sorted(w for w in shared_vocabulary if all(1 <=
genre_counts[g][w] <= 3 for g in GENRES))
36 shared_rare = pd.DataFrame([{'word': w, **{g: genre_counts[g][w] for
g in GENRES}} for w in shared_rare_words],
37                           columns=['word'] + GENRES)
38 display(Markdown('### Слова, встречающиеся во всех 4 жанрах и в
каждом не более 3 раз'))
39 display(shared_rare)
40 if shared_rare.empty: print('Таких слов в выбранных 40 песнях нет —
это допустимый результат.')
41 save_table(shared_rare, 'task3_shared_rare_FULL.csv')
43 top_categories =
category_totals.loc[category_totals.groupby('genre').per_1000_tokens.id
xmax()]
44 display(Markdown('**Вывод по выборке:**'))
45 for row in top_categories.itertuples():
46     print(f'{row.genre}: наиболее частая из заданных категорий —
{row.category}, '
47           f'{row.per_1000_tokens:.1f} упоминаний на 1000
обработанных токенов.')
```

27–28. Первый список: в каждом жанре отдельно оставляет слова с частотой от 1 до 3.

29–32. Показывает количество таких слов и первые 15 примеров на жанр; сохраняет полный список.

34. Пересекает словари: слова должны присутствовать во всех четырёх жанрах.

35. all проверяет, что частота в КАЖДОМ жанре от 1 до 3.

36–39. Создаёт и показывает таблицу общих редких слов с четырьмя счётчиками.

40–41. При пустом результате объясняет это; в любом случае сохраняет таблицу.

43. Для каждого жанра idxmax находит наиболее частую из трёх категорий.

44–47. Печатает категорию и относительную частоту.

Первый список: country — **372**, blues — **245**, folk — **370**, hip-hop — **629** слов. **Второй список:** всего **15** общих редких слов. Примеры фактического вывода:

Слово	Country	Blues	Folk	Hip-hop
alone	1	3	1	1
everything	1	1	2	1
morning	1	3	1	3
smile	2	2	2	1

Что могут спросить

Две трактовки редкости: показаны из-за неоднозначности PDF. Счётчики 3/3/3/3 подходят второму условию, хотя сумма 12. Счётчики 1/0/2/1 не подходят: слово отсутствует в жанре.

Распознаём ли смысл? Нет: heart бывает метафорой, fall — глаголом, spring — не сезоном. Это словарный поиск упоминаний, не контекстное снятие многозначности. **Почему не TF-IDF?** Требуются реальные количества слов; TF-IDF менял бы их смысл.

Можно ли обобщить на жанр? Только осторожно: 10 песен мало, припевы увеличивают частоты, нет контроля исполнителя/эпохи. Body_parts наиболее частая **среди заданных трёх категорий**; это не означает, что большинство слов песен относятся к телу. Редкость не равна важности: имена, сленг и опечатки тоже бывают редкими. Нужны больший корпус и контекстная проверка.

Задание 4 · Условие и чтение WiLI по строкам

Нужно: ≥500 примеров для 6–8 языков с близкой парой; 80/20 без дубликатов; сравнить очистку, char/word TF-IDF, NB/LR и fastText, ошибки и длины ввода. **Датасет найден:** WiLI-2018, зеркало alianoroozi/Language-Identification. Выбраны English, Russian, French, German, Spanish, Portuguese; близкая пара — последние два.

Блок 16, строки 1–27

```
01 WILI_BASE = ('https://raw.githubusercontent.com/alianoroozi/
Language-Identification/'
02               '9338c39f7dba99785bbedc7309013091f22d838c/wili-2018/')
03 LANGUAGES = {'eng': 'English', 'rus': 'Russian', 'fra': 'French',
04              'deu': 'German', 'spa': 'Spanish', 'por': 'Portuguese'}
05 ISO3_TO_ISO2 = {'eng': 'en', 'rus': 'ru', 'fra': 'fr', 'deu': 'de',
'spa': 'es', 'por': 'pt'}
07 def minimal_clean(text):
08     return re.sub(r'\s+', ' ', unicodedata.normalize('NFKC',
str(text))).strip()
10 def aggressive_clean(text):
12     return re.sub(r'\s+', ' ', ''.join(ch if ch.isalpha() or
ch.isspace() else ' '
13                                        for ch in
minimal_clean(text).lower())).strip()
15 assert aggressive_clean('Ә Қ Ғ Ң Ө Ү é ñ 123 !') == 'ə қ ғ ң ө ү é
ñ'
16 wili_rows = []
17 for split in ['train', 'test']:
18     x_path = fetch(WILI_BASE + f'x_{split}.txt', DATA / 'wili' /
f'x_{split}.txt')
19     y_path = fetch(WILI_BASE + f'y_{split}.txt', DATA / 'wili' /
f'y_{split}.txt')
20     from itertools import zip_longest
21     with open(x_path, encoding='utf-8-sig') as xf, open(y_path,
encoding='utf-8-sig') as yf:
22         for line_no, (text, label) in enumerate(zip_longest(xf,
yf)):
23             assert text is not None and label is not None, 'Число
строк текстов и меток различается'
24             label = label.strip()
25             if label in LANGUAGES:
26                 wili_rows.append({'text': minimal_clean(text),
'label': LANGUAGES[label],
27                                   'iso3': label, 'source_split':
split, 'source_line': line_no})
```

1–5. URL и таблицы кодов ISO3→имена и ISO3→ISO2; fastText использует короткие коды.

7–8. Minimal: NFKC и нормализация пробелов; регистр/цифры/пунктуация остаются.

10–13. Aggressive: lowercase; оставляет все Unicode-буквы и пробелы, прочее заменяет пробелами.

15. Проверяет сохранность ә/қ/ғ/ң/ө/ү и é/ñ. Не добавляет эти языки в обучение.

16–19. Для исходных train и test скачивает x-тексты и y-метки.

20–23. zip_longest читает соответствующие строки; assert обнаружит разную длину файлов.

24–25. Чистит перевод строки метки, отбирает только шесть языков.

26–27. Сохраняет текст, понятную метку, код и исходные split/номер строки.

Промежуточный результат: список словарей wili_rows; каждая строка — текст с меткой, а не отдельное слово. На следующей странице он превращается в таблицу и делится. Пример проверки очистки: `Ә Қ Ғ Ң Ө Ү é ñ 123 !` → `ə қ ғ ң ө ү é ñ`.

Почему это важно: `[a-z]` выбросил бы кириллицу и диакритику. Леммы и стоп-слова здесь не удаляются: служебные слова, буквы и окончания — полезные сигналы языка. Один объект WiLI часто является абзацем, а не одним предложением; короткий ввод проверяется отдельно.

Задание 4 · Балансировка и восемь обучений

Блок 16, строки 28–44

```
28 wili = pd.DataFrame(wili_rows)
29 wili['canonical'] = wili.text.map(aggressive_clean)
30 wili = wili[wili.canonical.str.split().str.len() >= 3]
31 label_counts = wili.groupby('canonical').label.nunique()
32 wili = wili[~wili.canonical.isin(label_counts[label_counts >
1].index)].drop_duplicates('canonical')
33
display(wili.label.value_counts().to_frame('available_unique_examples')
)
34 N_PER_LANGUAGE = 800
35 assert wili.label.value_counts().min() >= N_PER_LANGUAGE
36 language_data = pd.concat([group.sample(N_PER_LANGUAGE,
random_state=SEED)
37                            for _, group in wili.groupby('label')],
ignore_index=True)
38 lang_train, lang_test = train_test_split(language_data,
test_size=.2, random_state=SEED,
39                                         stratify=language_data.label)
40 assert set(lang_train.canonical).isdisjoint(lang_test.canonical)
41 assert set(lang_train.text).isdisjoint(lang_test.text)
42 assert len(lang_train) == 3840 and len(lang_test) == 960
43 display(pd.concat({'train': lang_train.label.value_counts(), 'test':
lang_test.label.value_counts()}, axis=1))
44 save_table(language_data.drop(columns='canonical'),
'task4_selected_data.csv')
```

28–30. Создаёт таблицу; aggressive-ключ для поиска повторов; исключает тексты короче 3 слов.

31–32. Убирает одинаковые ключи с разными метками, затем оставшиеся повторы.

33–37. Проверяет достаточность; выбирает по 800 на язык с фиксированным seed.

38–42. Стратифицированно делит 80/20, проверяет отсутствие общих ключей/текстов и точный размер.

43–44. Показывает числа, сохраняет выбранные примеры.

Блок 17, строки 1–22

```
01 LANG_LABELS = sorted(LANGUAGES.values())
02 language_models, language_predictions, language_scores = {}, {}, []
03 for cleaning, preprocessor in [('minimal', minimal_clean),
('aggressive', aggressive_clean)]:
04     for feature_type in ['char', 'word']:
05         for model_type in ['NB', 'LR']:
06             vectorizer = TfidfVectorizer(preprocessor=preprocessor,
lowercase=False,
07                 analyzer=feature_type, ngram_range=(1, 3) if
feature_type == 'char' else (1, 1),
08                 min_df=2, max_features=60000, sublinear_tf=True)
09             estimator = (MultinomialNB(alpha=.1) if model_type ==
'NB' else
10                          LogisticRegression(C=3, max_iter=1000,
random_state=SEED))
11             model = Pipeline([('tfidf', vectorizer), ('classifier',
estimator)])
12             model.fit(lang_train.text, lang_train.label)
13             name = f'{cleaning}/{feature_type}/{model_type}'
14             predicted = model.predict(lang_test.text)
15             language_models[name] = model
16             language_predictions[name] = predicted
17             language_scores.append({'model': name, 'cleaning':
cleaning, 'features': feature_type,
18                                     'classifier': model_type,
**evaluate(lang_test.label, predicted)})
19             print(name, 'готово', flush=True)
20 language_metrics = pd.DataFrame(language_scores)
21 display(language_metrics.sort_values('macro_f1', ascending=False))
22 save_table(language_metrics, 'task4_classical_metrics.csv')
```

1–5. Фиксирует порядок классов; циклы: 2 очистки × 2 признака × 2 модели.

6–8. Char: n-граммы 1–3; word: отдельные слова. min_df=2, максимум 60 000, logarithmic TF; регистр контролирует наша функция.

9–11. Создаёт NB(alpha=.1) либо LR(C=3), объединяет с TF-IDF.

12–16. fit на train, predict на test; сохраняет модель и ответы под именем эксперимента.

17–22. Считает и показывает метрики, сохраняет таблицу; flush сообщает прогресс сразу.

Выход split: каждый язык 640 train / 160 test; всего **3 840 / 960**. Все 8 моделей обучены на одинаковом наборе. Настройки заданы заранее; отдельной validation здесь нет. Исходные WiLI train/test объединены перед новым split: это учебный 80/20, не официальный benchmark.

Char может узнать сочетания букв незнакомого слова; word зависит от обученного словаря. NB использует условную независимость признаков и сглаживание alpha; LR — взвешенные оценки классов с регуляризацией. Соседние буквы не независимы буквально, но NB полезен как baseline. По Байесу P(язык|признаки) пропорциональна P(язык)×P(признаки|язык); наивная модель раскладывает последнее на произведение оценок признаков. Alpha не даёт нулевым частотам всё обнулить.

Задание 4 · FastText и общий результат

Блок 18, строки 1–34

```
01 import fasttext
02 FASTTEXT_SHA256 =
'8f3472cfe8738a7b6099e8e999c3cbfae0dcd15696aac7d7738a8039db603e83'
03 FASTTEXT_URLS = [
04     'https://dl.fbaipublicfiles.com/fasttext/supervised-models/
lid.176.ftz',
05     ('https://raw.githubusercontent.com/sherozshaikh/pdflangdetect/'
06      '068fe2f45464a8124617dfe1b58fad8cf82443b4/src/pdflangdetect/
_data/lid.176.ftz')]
07 ft_path = None
08 for url in FASTTEXT_URLS:
09     try:
10         ft_path = fetch(url, DATA / 'lid.176.ftz',
expected_sha256=FASTTEXT_SHA256)
11         break
12     except (requests.RequestException, ValueError) as exc:
13         print(f'Источник недоступен: {type(exc).__name__}. Пробуем
проверенное зеркало.')
14 if ft_path is None: raise RuntimeError('Не удалось загрузить
fastText. Нужен доступ к одному из источников.')
15 ft_model = fasttext.load_model(str(ft_path))
16 iso2_names = {ISO3_TO_ISO2[k]: v for k, v in LANGUAGES.items()}
18 def fasttext_predict(texts, with_confidence=False):
19     labels, confidences = [], []
20     for text in texts:
21         safe = minimal_clean(text).replace('\n', ' ')
22         label, probability = ft_model.predict(safe, k=1)
23         iso2 = label[0].removeprefix('__label__')
24         labels.append(iso2_names.get(iso2, f'other:{iso2}'))
25         confidences.append(float(probability[0]))
26     return (np.array(labels), np.array(confidences)) if
with_confidence else np.array(labels)
28 ft_pred = fasttext_predict(lang_test.text)
29 language_predictions['fastText lid.176'] = ft_pred
30 all_language_metrics = pd.concat([language_metrics, pd.DataFrame([
31     {'model': 'fastText lid.176', 'cleaning': 'minimal', 'features':
'pretrained',
32      'classifier': 'fastText', **evaluate(lang_test.label,
ft_pred)}])], ignore_index=True)
33 display(all_language_metrics.sort_values('macro_f1',
ascending=False))
34 save_table(all_language_metrics, 'task4_all_metrics.csv')
```

1–6. Задаёт ожидаемый SHA-256 и два источника одной модели lid.176.ftz.

7–14. Пробует официальный URL, затем зеркало; при полном провале останавливается.

15–16. Загружает готовые веса; переводит ISO2 в имена шести языков.

18–21. Для списка текстов создаёт накопители; чистит переносы.

22–25. Получает top-1 label/probability; снимает __label__; неизвестный код оставляет other:xx.

26. Возвращает метки, а по флагу — ещё и вероятности.

28–34. Прогнозирует тот же test; добавляет девятую строку и сохраняет метрики.

Блок 19, строки 12–18

```
12 PRIMARY_LANGUAGE_MODEL = 'minimal/char/LR'
13 primary_language_model = language_models[PRIMARY_LANGUAGE_MODEL]
14 primary_lang_pred = language_predictions[PRIMARY_LANGUAGE_MODEL]
15 print(classification_report(lang_test.label, primary_lang_pred,
labels=LANG_LABELS, zero_division=0))
16 show_confusion(lang_test.label, primary_lang_pred, LANG_LABELS,
PRIMARY_LANGUAGE_MODEL)
17 ft_cm_labels = LANG_LABELS + sorted(set(ft_pred) - set(LANG_LABELS))
18 show_confusion(lang_test.label, ft_pred, ft_cm_labels, 'fastText
lid.176 — all predicted labels')
```

12–14. Основной метод задан заранее: minimal/char/LR; берёт уже обученную модель и ответы.

15–18. Печатает отчёт и две confusion matrix; для fastText включает дополнительные предсказанные языки.

Часть общей таблицы; все 8+1 результаты сохранены в ноутбуке:

Метод	Accuracy	Macro F1
Minimal / char / NB	98.02%	0.9805
Minimal / char / LR	99.06%	0.9907
Minimal / word / NB	99.06%	0.9907
FastText lid.176	98.12%	0.9820

FastText не обучается здесь: это готовая компактная модель для 176 языков. Ответ вне шести считается ошибкой. Macro F1 всех методов усредняется по шести истинным классам. У fastText внешнее обучение, включая Wikipedia; возможное пересечение с WiLI ограничивает независимость сравнения. Узкая LR не объявляется лучшей на любых языках.

Задание 4 · Откуда ошибки и как создаётся тест длины

Блок 19, строки 20–36

```
20 lang_error_frame = lang_test[['text',
'label']].assign(predicted=primary_lang_pred)
21 lang_error_frame = lang_error_frame[lang_error_frame.label !=
lang_error_frame.predicted]
22 frequent_pairs = (lang_error_frame.groupby(['label',
'predicted']).size().rename('errors')
23                   .reset_index().sort_values('errors',
ascending=False))
24 display(frequent_pairs.head(3))
25 for row in frequent_pairs.head(3).itertuples():
26     example = lang_error_frame[(lang_error_frame.label == row.label)
&
27                                (lang_error_frame.predicted ==
row.predicted)].iloc[0].text
28     print(f'\n{row.label} → {row.predicted}: {row.errors} ошибок.
Пример: {example[:500]}')
29     if {row.label, row.predicted} == {'Spanish', 'Portuguese'}:
30         print('Возможная причина: близкая лексика, общий алфавит и
похожие короткие сочетания букв.')
31     else:
32         print('Проверить цитаты на другом языке, имена, общую
лексику и возможную ошибку разметки Wikipedia.')
33 if len(frequent_pairs) < 3:
34     print('На test встретилось меньше трёх типов ошибок;
дополнительные ошибки не выдумываем.')
35 save_table(lang_error_frame, 'task4_errors.csv')
36 save_table(frequent_pairs, 'task4_error_pairs.csv')
```

20–21. Добавляет ответы основной модели и оставляет ошибки.

22–24. Группирует истинный→предсказанный, считает и сортирует направления.

25–28. Для трёх частых пар выводит пример исходного текста.

29–34. Предлагает проверить близость языков либо цитаты/разметку. Если пар меньше 3, не выдумывает новые.

35–36. Сохраняет ошибки и их счётчики.

Вывод: Portuguese→English **4**, Russian→English **2**, French→English **1**. Пример Russian→English: “Thomson, Harry C.; Mayo, Lida. The Ordnance Department...” Английская библиография размечена по языку страницы: это может быть проблема метки, не букв.

Блок 20, строки 1–25

```
01 train_text_keys = set(lang_train.text.map(aggressive_clean))
02 train_sentence_keys = set()
03 train_short_keys = set()
04 for text in lang_train.text:
05     for sent in nltk.sent_tokenize(text):
train_sentence_keys.add(aggressive_clean(sent))
06     train_short_keys.add(aggressive_clean(' '.join(text.split()
[:3])))
07 stress_rows = []
08 for language, group in lang_test.groupby('label'):
09     examples = group.text.tolist()
10     for i, text in enumerate(examples):
11         sentences = nltk.sent_tokenize(text)
12         if not sentences: continue
13         variants = [('1–3 words', ' '.join(text.split()[:3]),
False),
14                     ('1 sentence', sentences[0], False)]
15         if len(sentences) >= 2:
16             variants.append(('2+ sentences', '
'.join(sentences[:2]), False))
17         elif len(examples) > 1:
18             other = nltk.sent_tokenize(examples[(i + 1) %
len(examples)])[0]
19             variants.append(('2+ sentences', sentences[0] + ' ' +
other, True))
20         for bucket, variant, paired in variants:
21             key = aggressive_clean(variant)
22             if not key or key in train_text_keys or key in
train_sentence_keys or key in train_short_keys: continue
23             stress_rows.append({'label': language, 'bucket': bucket,
'text': variant,
24                                 'paired_test_sentences': paired,
'word_count': len(variant.split())})
25 stress_data = pd.DataFrame(stress_rows).drop_duplicates(['bucket',
'text'])
```

1–6. Собирает ключи train-текстов, предложений и трёхсловных префиксов для исключения совпадений.

7–12. Перебирает test по языкам и предложениям; пустые пропускает.

13–14. Создаёт первые 3 слова и первое предложение.

15–19. Берёт 2 предложения; если второго нет — добавляет из следующего test-текста того же языка с paired=True.

20–22. Нормализует вариант; совпадения с train и пустые исключает.

23–25. Сохраняет метку, длину, группу и флаг склейки; убирает повторы вариантов.

Блок 19, строки 2–9: `pivot(...)` сопоставляет очистки; `aggressive - minimal` даёт ΔF1. Minimal лучше в 3 из 4 пар; для word/LR aggressive немного лучше. Различия малы и не проверялись на значимость. Нельзя утверждать «очистка всегда улучшает качество».

Задание 4 · Вычисление метрик длины и ответ пользователю

Блок 20, строки 26–57

```
26 stress_predictions = {name: model.predict(stress_data.text) for
name, model in language_models.items()}
27 stress_predictions['fastText lid.176'] =
fasttext_predict(stress_data.text)
28 stress_scores = []
29 for name, pred in stress_predictions.items():
30     for bucket in ['1–3 words', '1 sentence', '2+ sentences']:
31         mask = stress_data.bucket == bucket
32         if not mask.any(): continue
33         stress_scores.append({'model': name, 'bucket': bucket, 'n':
int(mask.sum()),
34             'mean_words': float(stress_data.loc[mask,
'word_count'].mean()),
35             **evaluate(stress_data.loc[mask, 'label'],
pred[mask.to_numpy()])})
36 stress_metrics = pd.DataFrame(stress_scores)
37 display(stress_metrics)
38 plot_order = ['1–3 words', '1 sentence', '2+ sentences']
39 stress_metrics.pivot(index='bucket', columns='model',
values='accuracy').reindex(plot_order).plot.bar(figsize=(14, 5))
40 plt.ylabel('Accuracy'); plt.ylim(0, 1.05); plt.xticks(rotation=0)
41 plt.legend(loc='lower right', fontsize=8); plt.tight_layout()
42 plt.savefig(RESULTS / 'task4_length_accuracy.png', dpi=150);
plt.show()
43 save_table(stress_metrics, 'task4_length_metrics.csv')
44 save_table(stress_data, 'task4_length_examples.csv')
46 def identify_language(text):
47     if not minimal_clean(text): raise ValueError('Введите непустой
текст')
48     probs = primary_language_model.predict_proba([text])[0]
49     classes =
primary_language_model.named_steps['classifier'].classes_
50     j = int(probs.argmax())
51     ft_label, ft_conf = fasttext_predict([text],
with_confidence=True)
52     return {'text': text, 'TFIDF_LR': classes[j],
'LR_probability_uncalibrated': float(probs[j]),
53             'fastText': ft_label[0],
'fastText_probability_uncalibrated': float(ft_conf[0])}
54 display(pd.DataFrame([identify_language(s) for s in [
55     'Сегодня очень хорошая погода.', 'The weather is wonderful
today.',
56     'Hoy hace muy buen tiempo.', 'Hoje o tempo está muito bom.',
57     'Il fait très beau aujourd’hui.', 'Heute ist das Wetter sehr
schön.']]))
```

26–27. Все 8 моделей и fastText получают одинаковые варианты stress_data.

28–32. Для модели и группы длины создаёт маску; пустую группу не оценивает.

33–37. Маской выбирает соответствующие истины/ответы, считает n, средние слова и метрики.

38–44. Фиксирует порядок групп, строит bar-график accuracy, сохраняет метрики и варианты.

46–47. identify_language отклоняет пустой вход.

48–50. LR возвращает вероятности; classes_ задаёт их порядок; argmax — индекс максимума.

51–53. Добавляет ответ fastText и обе некалиброванные оценки.

54–57. Запускает на шести примерах: ответы Russian/English/Spanish/Portuguese/ French/German.

Результат основной minimal/char/LR:

Вариант	n	Accuracy
Первые 3 слова	820	84.51%
Первое предложение	945	98.52%
Два предложения	960	99.06%

“Сегодня очень хорошая погода.” → **Russian**, LR≈**0.929**, fastText≈**0.991**. Это некалиброванные оценки: 0.9 не гарантирует 90% верных ответов.

Защита: короткому вводу не хватает сигналов. Варианты связаны, n различается, часть длинных склеена. «1–3 words» фактически 3 слова; NLTK-разбиение не настроено отдельно для каждого языка. LR выбирает из шести даже для неизвестного: казахского класса нет.

Финальная ячейка 21: `importlib.metadata.version()` фиксирует версии; `manifest={...}` собирает источники, хеши и размеры; `write_text(json.dumps(...))` пишет run_manifest.json; строки 11–18 выводят итог. Это журнал, не сохранение моделей в памяти. При перезапуске ядра нужны предыдущие fit. Notebook и его исходные результаты не изменены.